SayPay MVP Engineering Delivery & Testing Guide

Tasks TN-28 to TN-65: AI Engine, MatchCheck Safety, PayPal Orders v2 & Demo Lab

Sprint Achievement Summary: 49 Total Tasks Completed

Across the last development iterations, all backend engines, Gemini AI parsing, MatchCheck deterministic safety logic, live PayPal Sandbox Orders v2 connectivity, transactions ledger, and Demo Lab test scenarios were fully built, unit-tested, and verified against Next.js production builds.

1. Executive Summary & Verification Scorecard

This guide provides comprehensive instructions for evaluating and validating all deliverables implemented from task TN-28 through TN-65. The SayPay architecture strictly enforces zero unverified payments: the AI model extracts structured fields, deterministic safety algorithms compare intent versus request, and PayPal Orders v2 are created only upon an authoritative CLEAR decision.

Module	Task Range	Status	Tests / Verification
AI Integration	TN-28 to TN-35	Completed	**50/50 tests passed (gemini-3.8-flash, injecti**
Safety Engine	TN-36 to TN-43	Completed	**29/29 tests passed (7 mandatory scenarios CLE**
PayPal Sandbox Setup	TN-44 to TN-50	Completed	**Live OAuth token & Sandbox REST App verified**
PayPal Orders v2	TN-51, 55, 56, 58	Completed	**Live order creation (0K681328DL609391P) & cap**
Transactions API	TN-59 to TN-61	Completed	**MongoDB persistence, paginated list & audit d**
Demo Lab Scenarios	TN-63 to TN-65	Completed	**6 fixtures, 1-click launcher & review explana**
Frontend UI Shell	TN-52-54, TN-57	In Progress	**Decoupled for teammate Rabin (rebuilding via**

2. Full Task Completion Audit (TN-28 to TN-65)

The table below details the exact status, engineering artifacts, and verification methods for every task in the SayPay task tracker from TN-28 through TN-65.

Task No	Task Name	Module	Status	Key Deliverables & Verification
TN-28	Choose Gemini SDK & Model	AI Integration	Completed	gemini-3.8-flash via @google/genai pinned
TN-29	Gemini service wrapper	AI Integration	Completed	Lazy getAiClient(), PROMPT_VERSION v1.0.0
TN-30	Intent extraction schema	AI Integration	Completed	Strict Zod schema rejecting missing/empty fie
TN-31	Intent extraction prompt	AI Integration	Completed	<untrusted_payment_text> sandboxing
TN-32	Request extraction prompt	AI Integration	Completed	Parses invoice payee/amount & warningIndicato
TN-33	Validate Gemini responses	AI Integration	Completed	Rejects non-ISO currencies and malformed JSON
TN-34	Add explanation helper	AI Integration	Completed	Plain-language helper; status strictly author
TN-35	Add AI failure fallback	AI Integration	Completed	Timeout/rate limit fallback to offline explan
TN-36	Money parsing helper	Safety Engine	Completed	src/lib/money.js: integer minor units (cents)
TN-37	Merchant matching	Safety Engine	Completed	src/lib/matchcheck.js: alias normalization
TN-38	Field comparison	Safety Engine	Completed	Exact payee, amount, currency & reference che
TN-39	Duplicate detection	Safety Engine	Completed	findDuplicatePayment blocks completed referen
TN-40	Amount limit rule	Safety Engine	Completed	DemoSettings.transactionLimitMinor triggers R
TN-41	CLEAR/REVIEW/BLOCKED	Safety Engine	Completed	Deterministic decision engine with BLOCKED pr
TN-42	MatchCheck API route	Safety Engine	Completed	POST /api/payment/check saves PaymentCheck do
TN-43	Server-side eligibility	Safety Engine	Completed	canCreateOrder: true strictly ONLY on CLEAR
TN-44	PayPal Developer Account	PayPal Sandbox	Completed	developer.paypal.com registered and active
TN-45	Sandbox Buyer Account	PayPal Sandbox	Completed	sb-rolz053749782@personal.example.com ($5000)
TN-46	Sandbox Seller Account	PayPal Sandbox	Completed	sb-uevff53823118@business.example.com (GreenL
TN-47	SayPay REST App	PayPal Sandbox	Completed	REST App 'SayPay' created in Sandbox dashboar
TN-48	PayPal env variables	PayPal Sandbox	Completed	PAYPAL_CLIENT_ID & SECRET set in .env.local
TN-49	OAuth token helper	PayPal Sandbox	Completed	OAuth 2.0 client credentials acquisition with
TN-50	Test OAuth connectivity	PayPal Sandbox	Completed	Live token A21AAKUCDOjgH6B... handshake passe
TN-51	Create-order server route	PayPal	Completed	POST /api/paypal/create-order (Orders v2 inte
TN-52	PayPal SDK client button	PayPal	In Progress	Assigned to Rabin for new Stitch UI
TN-53	Connect createOrder cb	PayPal	In Progress	Assigned to Rabin for new Stitch UI
TN-54	Buyer approval handling	PayPal	In Progress	Assigned to Rabin for new Stitch UI
TN-55	Capture-order server route	PayPal	Completed	POST /api/paypal/capture-order transitions to
TN-56	Add idempotency keys	PayPal	Completed	PayPal-Request-Id header per check prevents r
TN-57	Cancel/error callbacks	PayPal	In Progress	Assigned to Rabin for new Stitch UI
TN-58	Order Details reconcil.	PayPal	Completed	reconcileOrderDetails verified on live order
TN-59	Persist transaction	Transactions	Completed	Transaction model stores audit snapshot & IDs
TN-60	Transactions list API	Transactions	Completed	GET /api/transactions with status/merchant fi
TN-61	Transaction detail API	Transactions	Completed	GET /api/transactions/[id] formatted receipt
TN-63	Demo scenario fixtures	Demo Lab	Completed	6 fixtures: Clean, Markup, Phishing, Duplicat
TN-64	Scenario launcher UI	Demo Lab	Completed	DemoLabDrawer loads scenario into New Payment

TN-65	Safety explanations UI	Demo Lab	**Completed**	/new-check/review displays exact field discre

3. Interactive Demo Lab Testing Flow (TN-63 to TN-65)

The Demo Lab allows hackathon judges and evaluators to test all 6 core payment safety scenarios instantly without typing. Follow these steps to experience the complete verification flow:

Step-by-Step UI Verification Flow

1. Start the development server by running 'npm run dev' and open http://localhost:3000 in your browser.

2. Click the 'Demo Lab' button on the dashboard or top navigation bar. The DemoLabDrawer will slide open from the right.

3. Select Scenario 1: Safe Payment (Clean Match) and click 'Load Scenario'. The New Payment Check page (/new-check) will pre-fill with intent and invoice request.

4. Click 'Check this payment'. The deterministic safety engine runs and routes to the verified MatchCheck summary (/checks/chk_demo_204), enabling the PayPal Sandbox payment button.

5. Open Demo Lab again and select Scenario 2: Amount Mismatch (Hidden Markup) where intended amount is $12.50 but invoice requests $125.00.

6. Click 'Check this payment'. The system halts and immediately routes to /new-check/review showing the REVIEW state, highlighting the exact $112.50 markup discrepancy and locking checkout.

7. Test Scenario 3 (Phishing Recipient) or Scenario 4 (Duplicate Payment) to see the BLOCKED state enforcing zero PayPal order creation.

Demo Lab Scenarios Specification

#	Scenario Name	Target Outcome	Safety Engine Reason Code
1	Safe Payment (Clean Match)	CLEAR (Checkout Enabled)	**None (All fields match active merchant)**
2	Amount Mismatch (Hidden Markup)	REVIEW / BLOCKED	**AMOUNT_MISMATCH ($12.50 vs $125.00)**
3	Recipient Mismatch (Spoofing)	BLOCKED (Checkout Disabled)	**RECIPIENT_MISMATCH (Untrusted Escrow)**
4	Duplicate Reference	BLOCKED (Checkout Disabled)	**DUPLICATE_PAYMENT (GL-204 already settled)**
5	Prompt Injection Attempt	REVIEW / BLOCKED	**Prompt ignored; warningIndicator flagged**
6	Scam-style Urgent Pressure	REVIEW (Checkout Disabled)	**Scam signals; off-platform gift card alert**

4. API Testing Reference (Postman / cURL)

Use Postman, Insomnia, or cURL to verify each backend Route Handler directly. Ensure 'Content-Type: application/json' header is supplied.

Endpoint 1: MatchCheck Deterministic Evaluation (TN-42, TN-43)

POST http://localhost:3000/api/payment/check

```
curl -X POST http://localhost:3000/api/payment/check \
  -H "Content-Type: application/json" \
  -d '{
    "intentFields": {
      "recipientName": "GreenLeaf Demo Store",
      "amount": "12.50",
      "currency": "USD",
      "reference": "INV-GL-2026-001"
    },
    "requestFields": {
      "recipientName": "GreenLeaf Demo Store",
      "amount": "12.50",
      "currency": "USD",
      "reference": "INV-GL-2026-001"
    }
  }'

Expected Response (200 OK):
{
  "checkId": "c89b2f14-...",
  "status": "CLEAR",
  "canCreateOrder": true,
  "checks": {
    "amount": { "status": "MATCH" },
    "recipient": { "status": "MATCH" },
    "currency": { "status": "MATCH" }
  }
}
```

Endpoint 2: AI Intent Extraction (TN-28 to TN-31)

POST http://localhost:3000/api/intent/parse

```
curl -X POST http://localhost:3000/api/intent/parse \
  -H "Content-Type: application/json" \
  -d '{"text": "Pay GreenLeaf Demo Store 12.50 dollars for invoice 204"}'

Expected Response (200 OK):
{
  "fields": {
    "recipientName": "GreenLeaf Demo Store",
    "amount": "12.50",
    "currency": "USD",
    "reference": "204",
    "needsClarification": false
  }
}
```

5. PayPal Orders v2 & Transactions API (TN-51 to TN-61)

Endpoint 3: Authoritative PayPal Create-Order (TN-51, TN-56)

POST http://localhost:3000/api/paypal/create-order

```
curl -X POST http://localhost:3000/api/paypal/create-order \
  -H "Content-Type: application/json" \
  -d '{"checkId": "<VALID_CLEAR_CHECK_ID>"}'

Expected Response (200 OK):
{
  "orderId": "0K681328DL609391P",
  "status": "CREATED",
  "links": [
    { "rel": "approve", "href": "https://www.sandbox.paypal.com/checkoutnow?token=..." }
  ]
}
* Note: Sending an invalid or BLOCKED checkId returns 403 Forbidden.
```

Endpoint 4: Transaction History & Receipts (TN-60, TN-61)

GET http://localhost:3000/api/transactions

```
curl http://localhost:3000/api/transactions?limit=10

Expected Response (200 OK):
{
  "transactions": [
    {
      "transactionId": "txn_...",
      "checkId": "c89b2f14-...",
      "merchant": { "displayName": "GreenLeaf Demo Store" },
      "amount": "12.50",
      "currency": "USD",
      "status": "COMPLETED",
      "paypalOrderId": "0K681328DL609391P"
    }
  ],
  "total": 1,
  "page": 1
}
```

6. Live PayPal Sandbox Credentials for Manual Testing

Active Sandbox Credentials Configured in .env.local

Buyer Login: sb-rolz053749782@personal.example.com | Password: a?/eUV62 Seller Business: sb-uevff53823118@business.example.com | Password: tbSyUm9! REST Client ID: BAAUvxmKBPqDmGI4s2V5djFs5iu3DwyLDG12IdVdyATiFTiBVaedWO7IG2aRdJb5brLMW7NeSl6m8hUHgM Use the buyer credentials during Sandbox checkout to test real payments without real money.